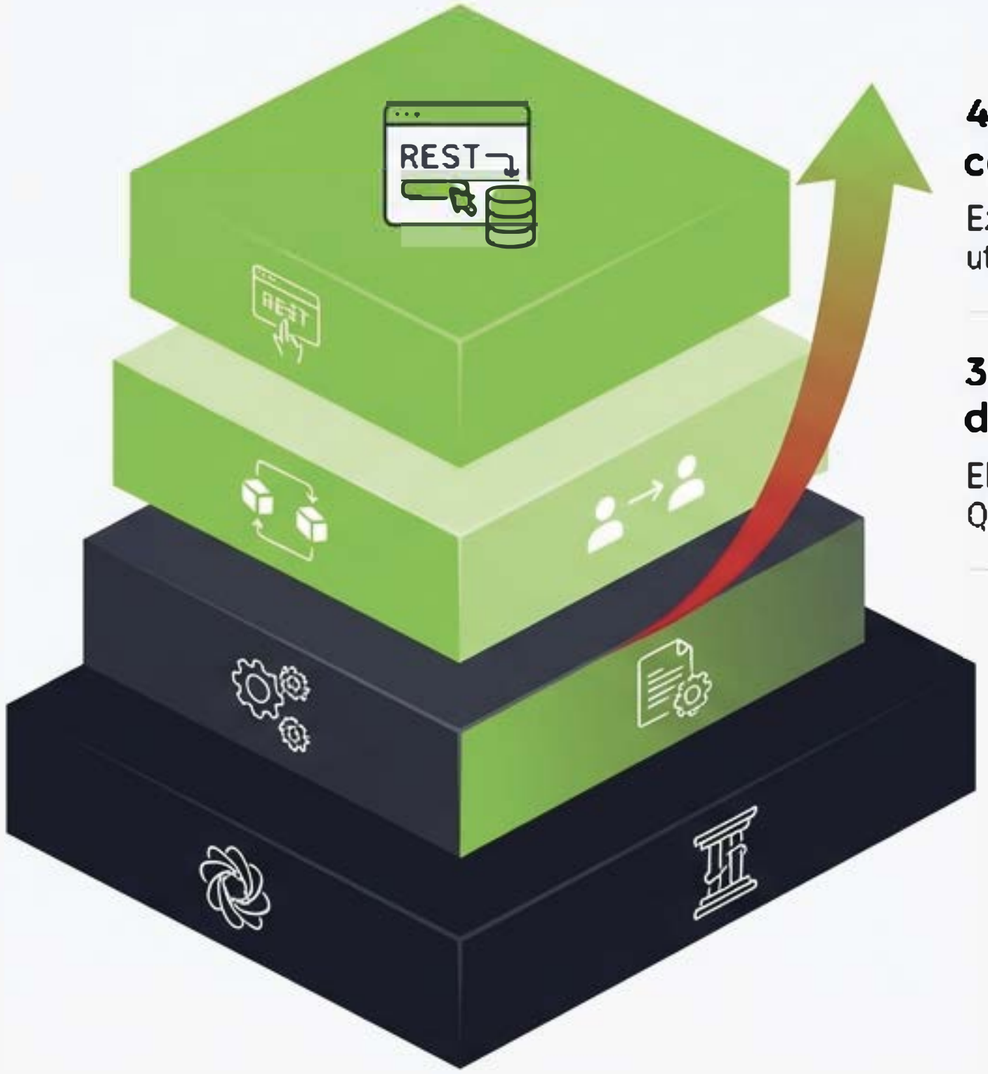
Desarrollo de Microservicios Cloud-Native con Quarkus Quarkus

Evolución y Arquitectura Empresarial

La Arquitectura del Curso

CLOUDNATIVE EVOLUTION



4. La Superficie: Desarrollo de APIs REST con RESTEasy (JAX-RS).

Exposición de servicios escalables y eficientes utilizando estándares JAXRS modernos.

`@Path("/api") @GET / @POST`

3. El Núcleo (La Piedra Rosetta): Inyección de Dependencias con ArC (CDI) vs. Spring.

El puente de traducción técnica: comparativa de CDI en Quarkus frente a la inyección en Spring.

`@Inject(CDI) vs. @Autowired(Spring)`

2. El Entorno: Configuración de la aplicación con SmallRye Config.

Externalización y gestión dinámica de la configuración en entornos distribuidos.

`application.properties / YAML`

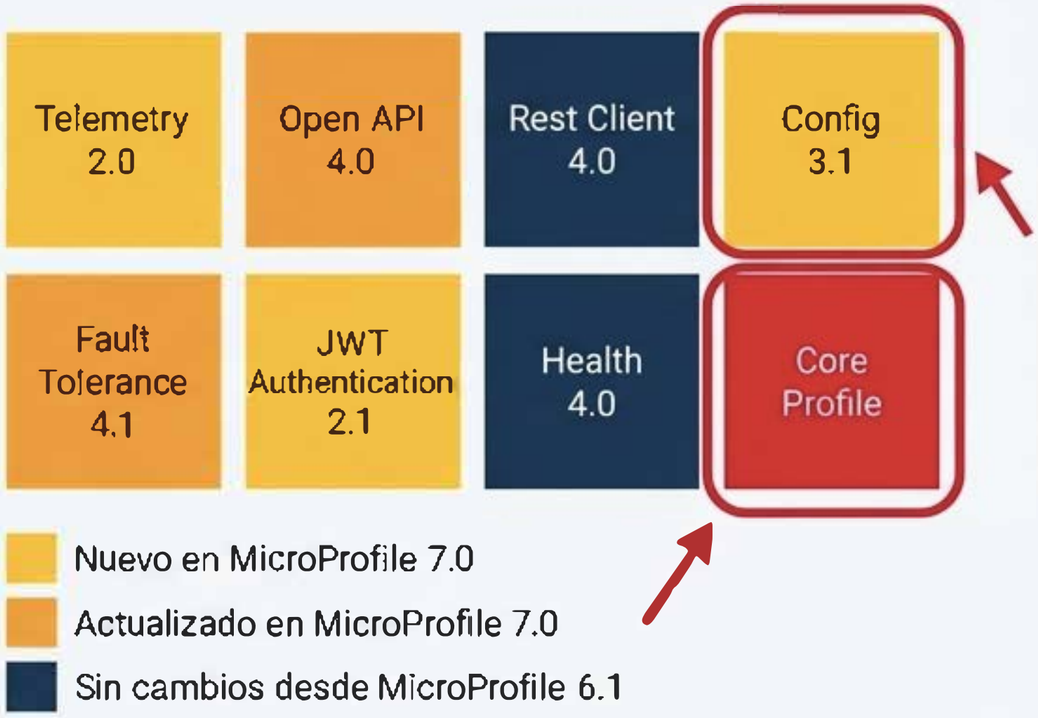
1. El Estándar: MicroProfile 7.0 y el ecosistema Jakarta EE.

La base fundamental para aplicaciones empresariales modernas y estandarizadas.

La Base Normativa: MicroProfile 7.0

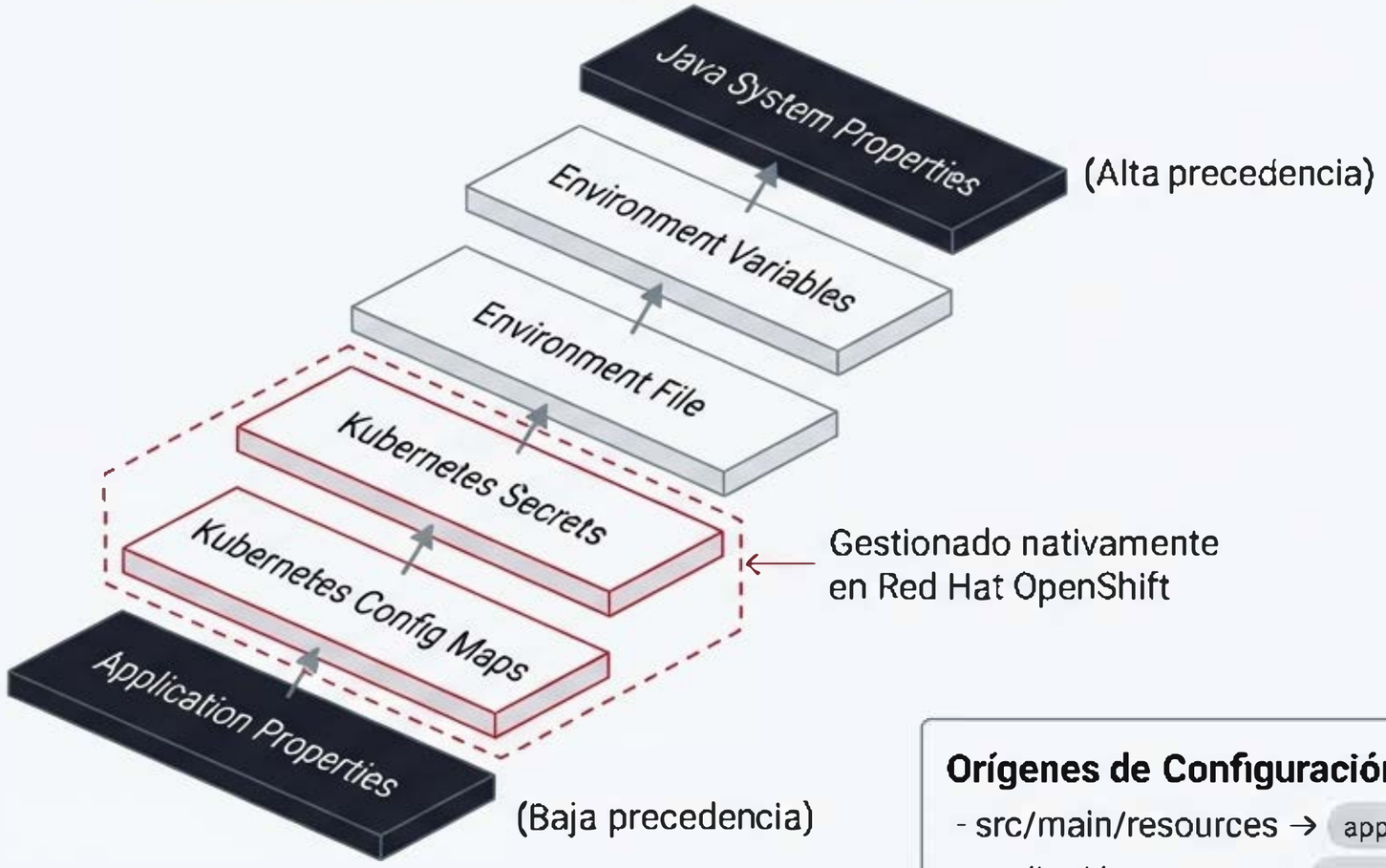
MicroProfile 7.0 es el estándar principal para microservicios Java empresariales.

- **Jakarta EE 10 Core Profile:** Ahora especificado como dependencia base, simplificando la integración.
- **MicroProfile Telemetry 2.0:** Reemplaza de manera integral a MicroProfile Metrics.
- **Config 3.1 & Core Profile:** Los pilares para la configuración y estructura base de nuestros microservicios.
- **Soporte Moderno:** Permite certificación en Java SE 11 y superior.



SmallRye Config: El Orden de Precedencia

En un entorno cloud-native, la configuración debe inyectarse desde el exterior. SmallRye resuelve conflictos aplicando una precedencia estricta (de mayor a menor):



Orígenes de Configuración Locales:

- src/main/resources → `application.properties`
- src/test/resources → `Tests`

La Piedra Rosetta: Traduciendo tu Experiencia

Tu conocimiento previo es tu mayor ventaja. Así se mapean los conceptos fundamentales en 2026:

Ecosistema Quarkus (Jakarta EE / MicroProfile)		Ecosistema Spring
ArC (CDI - Context and Dependency Injection) `ArC (CDI)`	**Inyección de Dependencias (El Contenedor IoC)**	Spring DI (@Component, @Autowired) `Spring DI (IoC Container, @Component, @Autowired, etc.)`
RESTEasy (JAX-RS) `RESTEasy (JAX-RS)`	**Capa Web (Controladores REST)**	Spring Web MVC (Spring REST Controllers) `Spring Web MVC (Spring REST Controllers)`
Jackson (mapea objetos Java hacia y desde JSON) `Jackson`	**Procesamiento de JSON**	Jackson (integrado por defecto en Spring Boot) `Jackson (ya integrado en Spring Boot)`

ArC (CDI): Descubrimiento de Beans

A diferencia de los frameworks tradicionales que escanean el classpath en tiempo de ejecución, el contenedor de inyección de dependencias ArC descubre las clases durante el proceso de compilación a través de cuatro mecanismos principales:

1. Anotaciones de Ciclo de Vida

Clases de la aplicación marcadas explícitamente (ej. @ApplicationScoped).

`@ApplicationScoped`

2. Descriptor XML

Dependencias declaradas en el archivo beans.xml.

`beans.xml`

3. Índice Jandex

Clases referenciadas pre-indexadas para lectura ultrarrápida.

4. Propiedades de Aplicación

Clases referenciadas específicamente específicamente mediante la propiedad `quarkus.index-dependency` en el `application.properties`

Gestión de Ciclo de Vida: Traduciendo Alcances (Scopes)

Comparativa de cómo se gestionan las instancias entre Jakarta EE (Quarkus) y Spring Framework.

Ecosistema Quarkus (Jakarta EE / ArC)		Ecosistema Spring	
Una instancia por aplicación		Una instancia por aplicación	
`@ApplicationScoped`	(Reutiliza la instancia para todos los puntos de inyección)	`@Scope("singleton")`	(Valor por defecto en el contenedor IoC)
Una instancia por petición HTTP		Una instancia por petición HTTP	
`@RequestScoped`		`@RequestScope`	
Una instancia por sesión de usuario		Una instancia por sesión de usuario	
`@SessionScoped`	(Gestionado por el objeto HttpSession al usar la extensión quarkus-undertow)	`@SessionScope`	
Una instancia cada vez que se inyecta		Una instancia cada vez que se inyecta	
`@Dependent`	(El ciclo de vida se ata al objeto padre que lo inyecta)	`@Scope("prototype")`	

Optimizando ArC: Buenas Prácticas y Limitaciones

Para mantener tiempos de arranque sub-segundo, ArC tiene restricciones intencionales frente a CDI tradicional:

1. Evitar Java Reflection (Inyección por Constructor)

Práctica recomendada: No usar campos privados. La inyección por constructor es superior y elimina la necesidad de reflexión.

```
@Inject private GreeterService greeter;
```

```
public GreeterBean(GreeterService greeter) {
    this.greeter = greeter;
}
```

2. Restricciones Arquitectónicas

- **beans.xml**: ArC ignora su contenido para configurar interceptors, decorators o alternatives.
- **BeanManager**: Implementa un subconjunto específico de métodos (getBeans(), resolve(), fireEvent(), etc.).
- **No Soportados**: @ConversationalScoped, extensiones portables, passivization, y métodos interceptores en superclases.

Construyendo la Superficie: JAX-RS vs Spring Web

Definiendo la ruta base y los verbos HTTP del endpoint:

		JAX-RS	Spring
1.	Ruta del Recurso:	`@Path("/resource")` (A nivel de clase o método)	`@RequestMapping("/resource")`
2.	Endpoint GET:	`@GET`	`@GetMapping` `@RequestMapping(method = GET)`
3.	Endpoint POST:	`@POST`	`@PostMapping` `@RequestMapping(method = POST)`

4.	Endpoint PUT / DELETE / HEAD:	`@PUT`
		`@DELETE`
		`@HEAD`
			`@PutMapping`
			`@DeleteMapping`
			`/ etc.`

Negociación de Contenido en Endpoints

La clase **javax.ws.rs.core.MediaType** especifica los valores permitidos para los tipos de medios de internet.

Tipo de Petición
(Lo que el endpoint acepta):

JAX-RS: @Consumes("application/json")

Spring: @PostMapping(consumes = "application/json")

Tipo de Respuesta
(Lo que el endpoint devuelve):

JAX-RS: @Produces("application/json")

Spring: @GetMapping(produces = "application/json")

Extrayendo Parámetros de la Petición

	JAX-RS	Spring
Segmento de la URI (Path):	`@PathParam("id")`	`@PathVariable("id")`
Parámetro de Query (?q=valor):	`@QueryParam("q")`	`@RequestParam("q")`
Cabecera HTTP (Header):	`@HeaderParam("h")`	`@RequestHeader("h")`
Campo de Formulario (application/x-www-form-urlencoded):	`@FormParam("f")`	`@RequestParam("f")` ó `@ModelAttribute`

Inyección de Contexto HTTP Avanzado

Cuando necesitas acceso directo a los objetos subyacentes del servidor web, JAX-RS provee una única y poderosa anotación.

La Anotación @Context (JAX-RS)

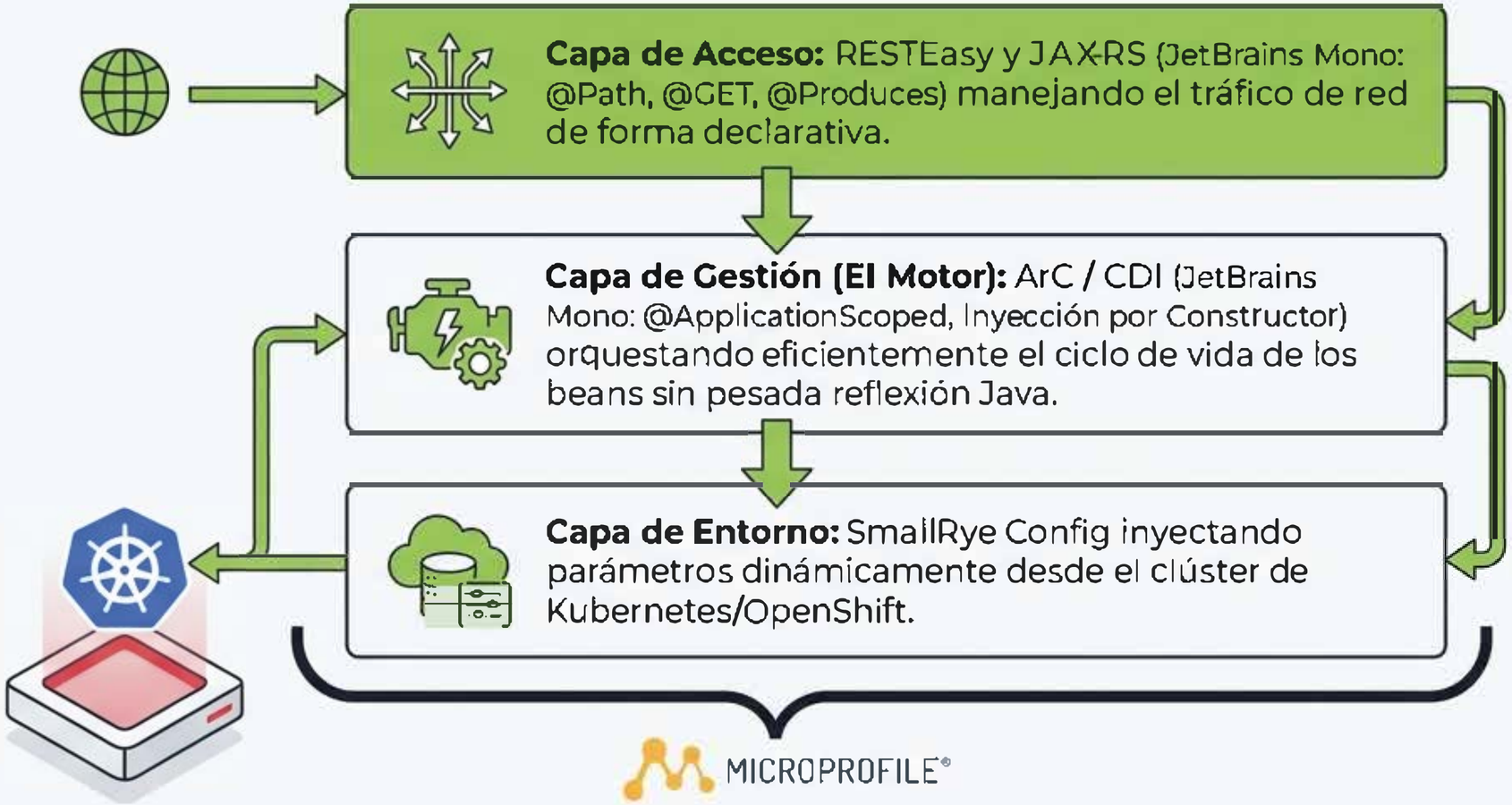
Permite retornar el contexto para la petición HTTP entrante. Se utiliza para inyectar variables a nivel de clase o directamente en los métodos.

- Retorna objetos como: `HttpServletRequest`, `SecurityContext`, o `UriInfo`.

Equivalencia en Spring Framework:

En Spring, esto se logra permitiendo el acceso al contexto inyectando los objetos (`HttpServletRequest`, `HttpServletResponse`, `Principal`) directamente en los parámetros del método, o utilizando `@Autowired` a nivel de clase.

El Ecosistema Cloud-Native Unificado



Todo respaldado por el rendimiento y la estandarización de MicroProfile 7.0.